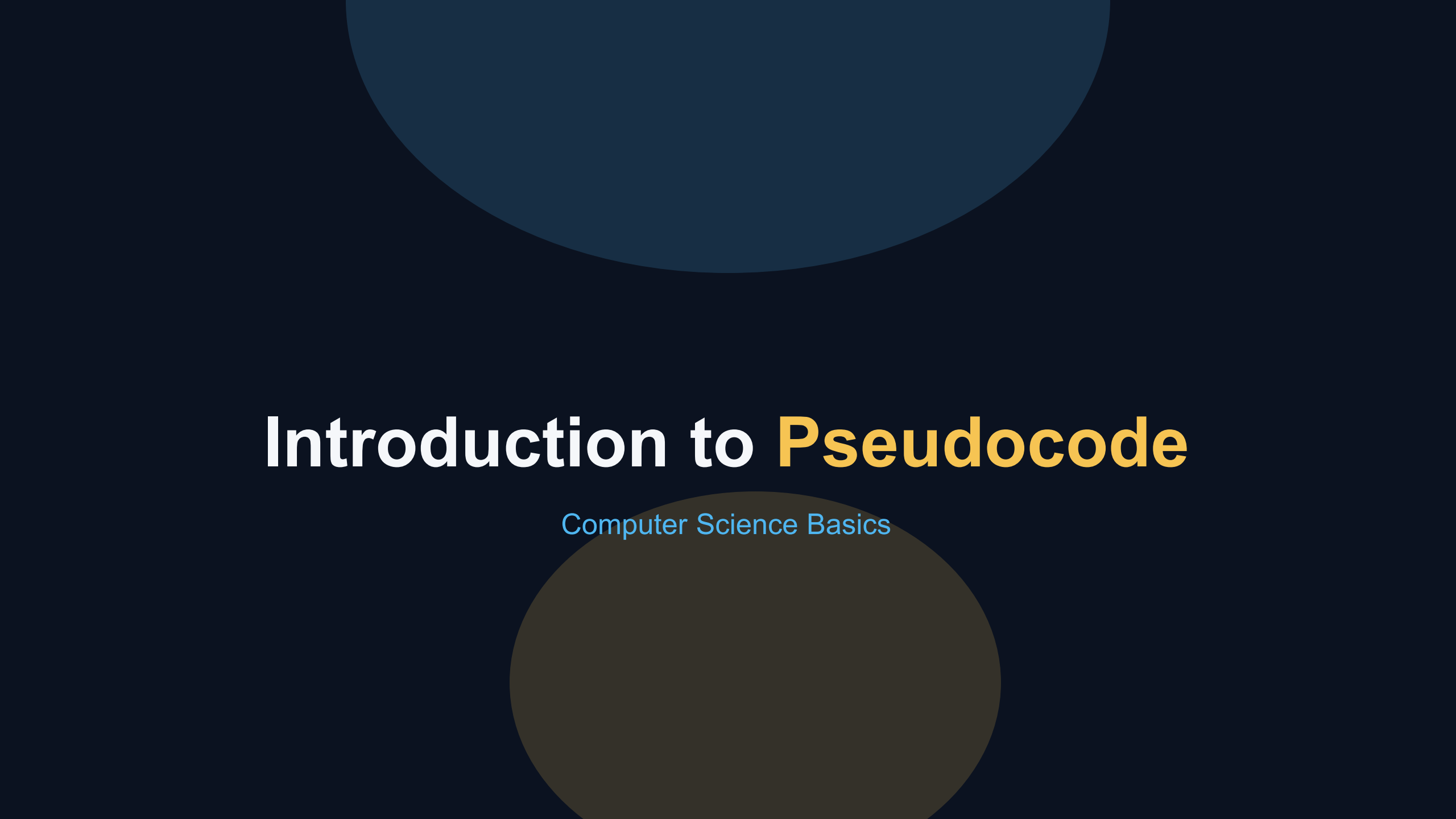




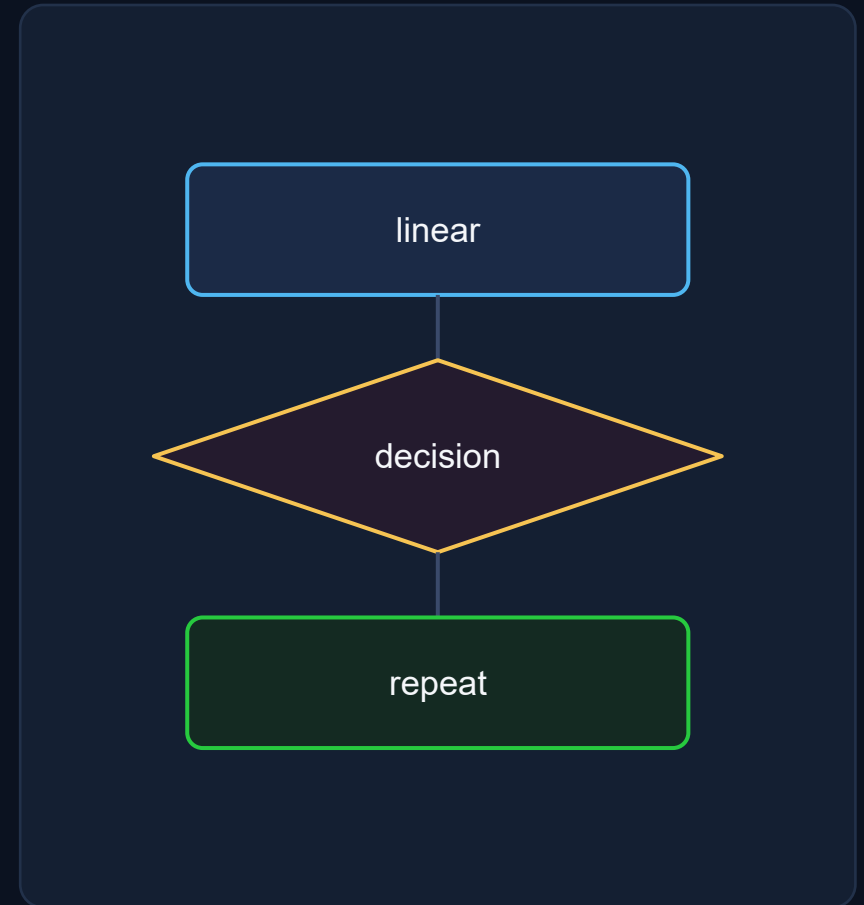
Introduction to Pseudocode

Computer Science Basics

GETTING STARTED

What is Pseudocode?

- Pseudocode is a language used to describe the steps of an algorithm.
- It borrows structures from real programming languages (Pascal, C/C++, BASIC, etc.), but it is meant to be read by people, not executed by computers.
- Implementation details are usually left out — such as variable declarations or language-specific syntax.
- Pseudocode can mix plain natural-language phrases with compact mathematical expressions that don't exist in real programming languages.



GETTING STARTED

Core Concepts

The concepts found in pseudocode algorithms are similar to those in real programming languages:

1

Instructions

2

Variables

3

Constants

4

Operations

5

Expressions

Many variants of pseudocode exist. This deck uses the variant found in Romanian Computer Science Baccalaureate exam problems — the same one taught in high-school algorithm courses.

Variables & Constants

- **Variables are never declared.**
- Their names follow the usual identifier rules: letters, digits, and the underscore character — and a name can never start with a digit.
- **Constants are literal values:**
- integers and reals, characters delimited with a single quote ('), and strings delimited with single (') or double (") quotes.
- Mathematical constants, such as π , may also appear directly in expressions.



```
// examples  
age ← 15    // integer  
gpa ← 3.85  // real  
name ← 'Alex' // string  
grade ← 'A'  // character
```

Data Operations

Arithmetic Operations

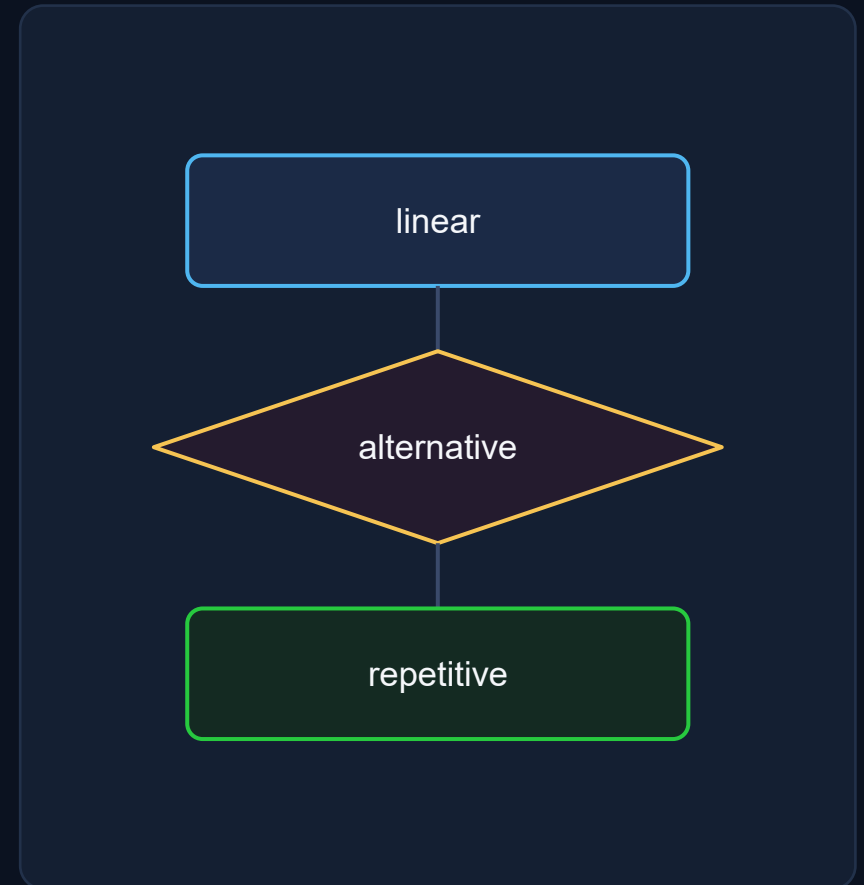
- Addition: $a + b$ — integer or real, depending on context
- Subtraction: $a - b$
- Multiplication: $a * b$
- Division: a / b — always real; for the integer quotient use the floor: $[a/b]$
- Remainder: $a \% b$ — integer operands only, integer result
- Some variants use DIV and MOD instead. Exponentiation and square root may also appear.

Relational & Logical Operations

- Relational: $=, \neq, <, \leq, >, \geq$
- Operands are usually numbers; the result is a boolean value (true or false).
- Logical operations: NOT, AND, OR — combine or invert boolean values.

Instructions: **The Basics**

- Instructions are the parts of an algorithm that actually have an effect — they change variable values, read or display data, repeat actions, and so on.
- Usually each instruction is written on its own line (or several, for complex ones).
- To save space, two or more simple instructions can share a line, separated by ;
- **Every algorithm can be built from three kinds of structure:**
- linear, alternative (decision), and repetitive — with the repetitive structure appearing in four forms, covered later in this deck.



Reading, Writing & Assignment

The three instructions every pseudocode algorithm starts with.

DEMO 1

Reading Data

- **read <variable list>**
- <variable list> is a sequence of variables separated by commas.
- Successive values are taken from the keyboard, in order, and stored in the listed variables.
- A read instruction is usually paired with notes about the expected data — its type, valid range, and so on.



```
read a, b, c
```

DEMO 2

Writing Data

- **write** <expression list>
- <expression list> is a sequence of expressions separated by commas.
- The expressions are evaluated in order, and their results are printed to the screen.



```
write 'a + b = ', a + b
```

```
// if a = 3 and b = 4:
```

```
// a + b = 7
```

DEMO 3

The **Assignment** Instruction

- **<variable> ← <expression>**
- The expression is evaluated first, and the resulting value is then stored in the variable.
- This is the workhorse instruction behind every calculation in a pseudocode algorithm.



```
a ← 0
```

```
S ← a + b
```

```
i ← i + 1
```

The Alternative Structure

- **Execution order:**
- Evaluate <condition>.
- If it is true, execute <instructions1>, then move on to the next instruction.
- If it is false, execute <instructions2> — when an else branch exists — then move on to the next instruction.



```
┌ if <condition> then  
|   <instructions1>  
| else  
|   <instructions2>  
└
```



```
┌ if <condition> then  
|   <instructions1>  
└
```

Repetitive Structures

Four ways to repeat a sequence of instructions.

FOR

known number of steps

WHILE

unknown steps, initial test

DO . . . WHILE

unknown steps, final test

REPEAT . . . UNTIL

unknown steps, final test

The **FOR** Loop



```
┌ for <var> ← <start>, <end>, <step> do
|   <instructions>
└
```

- start, end and step are expressions that evaluate to (usually) integers; step is 1 or -1, and defaults to 1 when omitted.
- The variable takes successive values — increasing if step = 1, decreasing if step = -1 — from start to end, running the instructions each time.
- If step = 1 and start > end (or step = -1 and start < end), the instructions never run.

Demo — sum & product of the first n natural numbers



```
S ← 0; P ← 1
┌ for i ← 1, n do
|   S ← S + i
|   P ← P * i
└
write S, ' ', P
```

The WHILE Loop



```
┌ while <condition> do  
|   <instructions>  
└─
```

- Evaluate <condition>.
- If true, run the instructions, then go back to step 1.
- If false, move on to the next instruction in the algorithm.
- **Important:**
 - if the condition is false from the start, the instructions never run.

Demo — sum of the digits of a natural number



```
read n  
S ← 0  
┌ while n ≠ 0 do  
|   S ← S + n % 10  
|   n ← [n / 10]  
└─  
write S
```

The DO...WHILE Loop



```
┌ do
|   <instructions>
└ while <condition>
```

- Run the instructions.
- Evaluate <condition>.
- If true, go back to step 1. If false, move on to the next instruction.
- **Important:**
 - even if the condition is false from the start, the instructions have already run once.

Demo — number of digits of a natural number



```
read n
cnt ← 0
┌ do
|   cnt ← cnt + 1
|   n ← [n / 10]
└ while n ≠ 0
write cnt
```

The REPEAT...UNTIL Loop



```
┌ repeat
|   <instructions>
└ until <condition>
```

- Run the instructions.
- Evaluate <condition>.
- If false, go back to step 1. If true, move on to the next instruction.
- **Important:**
- even if the condition is true from the start, the instructions have already run once.

Demo — number of digits of a natural number

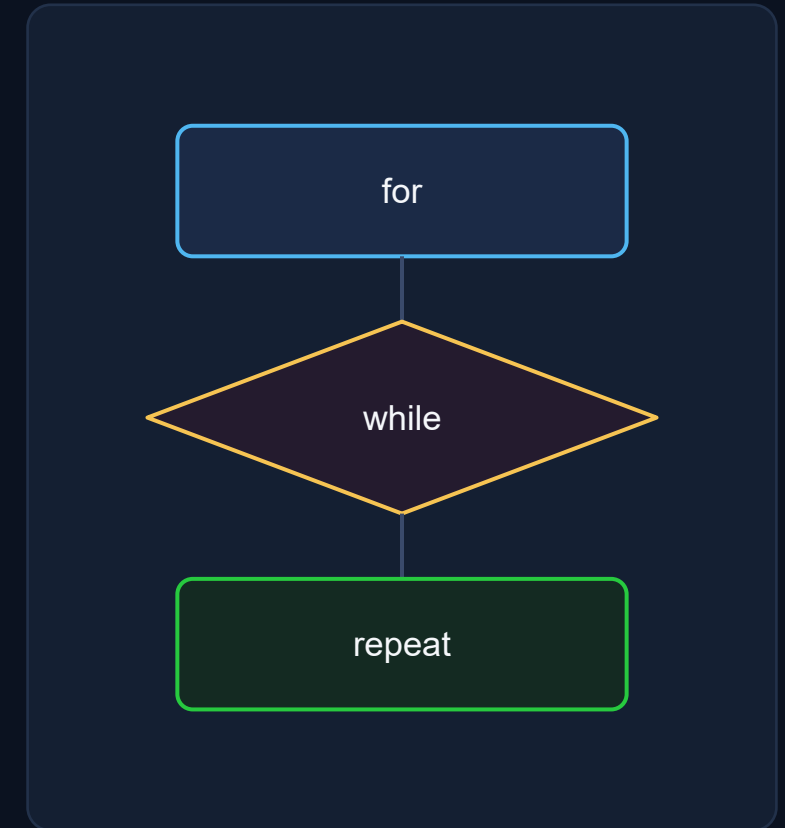


```
read n
cnt ← 0
┌ repeat
|   cnt ← cnt + 1
|   n ← [n / 10]
└ until n = 0
write cnt
```

REPETITIVE STRUCTURES

Equivalence of Repetitive Structures

- Many exercises give you an algorithm and ask for an equivalent one that uses a different repetitive structure.
- **Two algorithms are equivalent**
- if, for every set of valid input data, they produce exactly the same results.
- The next slides walk through the standard transformation rules between FOR, WHILE, DO...WHILE, and REPEAT...UNTIL.



STRUCTURE EQUIVALENCE

FOR → WHILE

Before



```
┌ for i ← a, b do  
|   <instructions>  
└─
```

Equivalent



```
i ← a  
┌ while i ≤ b do  
|   <instructions>  
|   i ← i + 1  
└─
```

STRUCTURE EQUIVALENCE

FOR → DO...WHILE

Before

```
┌ for i ← a, b do  
|   <instructions>  
└─
```

Equivalent

```
┌ i ← a  
| if i ≤ b then  
| ┌ do  
| |   <instructions>  
| |   i ← i + 1  
| └ while i ≤ b  
└─
```

STRUCTURE EQUIVALENCE

FOR → REPEAT...UNTIL

Before

```
┌ for i ← a, b do  
|   <instructions>  
└─
```

Equivalent

```
┌ i ← a  
└ if i ≤ b then  
  ┌ repeat  
  │   <instructions>  
  │   i ← i + 1  
  └ until i > b  
  └─
```

STRUCTURE EQUIVALENCE

WHILE → DO...WHILE

Before



```
┌ while <condition> do  
│   <instructions>  
└─
```

Equivalent



```
┌ if <condition> then  
│ ┌ do  
│ │   <instructions>  
│ └ while <condition>  
└─
```

STRUCTURE EQUIVALENCE

WHILE → REPEAT...UNTIL

Before



```
┌ while <condition> do  
|   <instructions>  
└─
```

Equivalent



```
┌ if <condition> then  
| ┌ repeat  
||   <instructions>  
| └ until NOT <condition>  
└─
```

STRUCTURE EQUIVALENCE

REPEAT...UNTIL → WHILE

Before



```
┌ repeat  
|   <instructions>  
└ until <condition>
```

Equivalent



```
<instructions>  
┌ while NOT <condition> do  
|   <instructions>  
└
```

Questions?

Next up: elementary algorithms — digit processing, divisibility, and sequence problems.

Thank you for participating!